Project: To-Do List API **CR ID:** CR-002 — API Idempotency **All statuses below are initial. No test has been executed.**

Test ID	Requirement ID	Test Scenario	Expected Result	Status
TC-002-01	FR-05	`POST /api/tasks` with no `Idempotency-Key` header	`201 Created`; behaves exactly as current (unmodified) behavior	Not Executed
TC-002-02	FR-01, FR-02	`POST /api/tasks` with a new `Idempotency-Key` and a valid body	`201 Created`; task is persisted; an idempotency record is stored for the key	Not Executed
TC-002-03	FR-03	Repeat TC-002-02's exact request (same key, same body)	`201 Created` with the **same** task id and data as TC-002-02; database shows only one `TASK` row for this request	Not Executed
TC-002-04	FR-04	Reuse the key from TC-002-02 with a different body	`409 Conflict`; no new task created; original task from TC-002-02 unchanged	Not Executed
TC-002-05	FR-04	Inspect the `409` response body from TC-002-04	Body follows the project's standard `ErrorResponse` shape and names the idempotency key conflict	Not Executed
TC-002-06	FR-02, FR-06	**Concurrency:** fire two identical `POST /api/tasks` requests simultaneously using the same new `Idempotency-Key` and the same body	Exactly one `TASK` row is created; both responses return the same task id	Not Executed
TC-002-07	FR-01	`POST /api/tasks` with an `Idempotency-Key` but an otherwise invalid body (e.g. missing `title`)	`400 Bad Request`; no idempotency record is stored (validation runs first)	Not Executed
TC-002-08	FR-07	Create an idempotency record, advance the clock/config past the retention window, run the cleanup job	Expired record is removed; a request reusing that same key afterward is treated as brand new	Not Executed
TC-002-09	— (regression)	Run the existing Playwright `tasks.spec.ts` suite unmodified (no `Idempotency-Key` sent by any existing test)	All existing test cases continue to pass	Not Executed
TC-002-10	— (data integrity)	After TC-002-03, inspect the `TASK` table directly	Exactly one row exists for the idempotent request; no orphaned or duplicate rows	Not Executed

Notes

- TC-002-06 should be implemented with genuinely parallel HTTP calls (not sequential), to validate the unique-constraint race behavior described in `TECHNICAL_DESIGN.md` §6, not just the simpler sequential-replay case already covered by TC-002-03.
- TC-002-08 depends on the retention/cleanup mechanism's final design (shared with CR-003's scheduler) and may need to use a configurable short retention window in the test environment rather than the 24-hour production default.